cv-lab

Local computer vision workspace — architecture and code reference

Oussama Boukhachem

28 October 2026 · v1.0

1. What it is

A single local application that watches a webcam and explains what it sees. It runs on one laptop GPU, keeps every frame on the machine, and needs no cloud service or API key.

Five capabilities share one frame pipeline: person detection and tracking, hand landmarks with held-gesture recognition, facial expression scoring, metric distance to any point, and a live 3D point cloud. A local vision-language model adds a short sentence about the scene. Everything is exposed on one web page at `127.0.0.1:8099`.

The design goal is honesty about limits: distance is approximate, expression is not mood, and glass defeats monocular depth entirely. Those limits are documented in section 6 rather than hidden behind confident interface copy.

2. Architecture

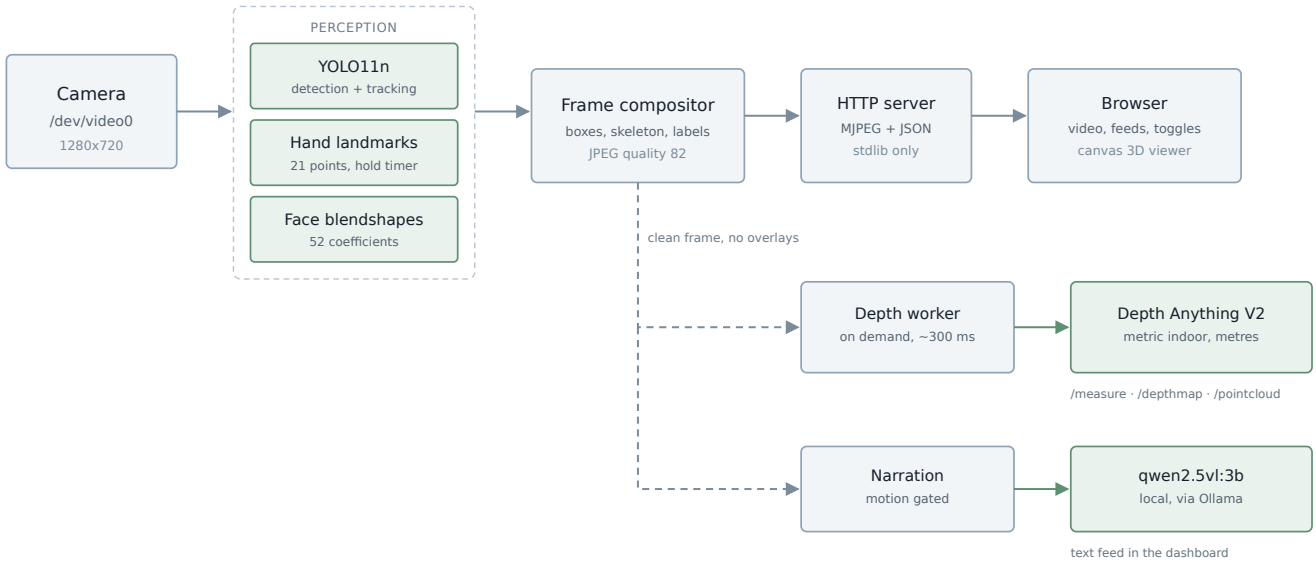


Figure 1. One capture thread feeds all models. Depth and narration read the same overlay-free frame, and only run when enabled.

3. Modules and functions

webui.py — server, capture loop, workers, page

Function	Responsibility
`_make_stop_frame()`	Renders the placeholder shown while the camera is stopped.
`describe()`	Posts one JPEG to Ollama and returns the sentence plus latency.
`add_event()`	Pushes a narration line into the shared feed.
`add_gesture()`	Records a gesture that completed its hold.
`vision_loop()`	Owns the camera. Reads frames, tracks, runs hands and face, draws, encodes, publishes. Handles stop/start and camera release.
`depth_loop()`	Runs only while a depth feature is on. Produces the metre map and heatmap, throttled to a few updates per second.
`vlm_loop()`	Watches for pixel change, then asks the model for a sentence. Yields whenever depth is active.
`Handler._send()`	Writes a full response with the headers the UI depends on.
`Handler.do_POST()`	Feature toggles and start/stop.
`Handler.do_GET()`	Routes page, stream, snapshot, events, depth, cloud, measure.
`Handler.log_message()`	Silences per-request logging.
`main()`	Parses options, starts the three workers, serves until interrupted.

gesture.py — hands and held gestures

Function	Responsibility
`HandTracker.__init__()`	Loads the MediaPipe hand model and sets the hold duration.
`HandTracker.detect()`	Returns hands with 21 landmarks, finger states, gesture name and hold progress.
`HandTracker.reset()`	Clears hold state when the camera stops, so nothing fires stale.
`HeldGesture.update()`	Accumulates how long the same gesture has persisted; returns a fire event once.
`_straightness()`	Chord over segment length for one finger chain: near 1 is extended, low is curled.
`_name_from_fingers()`	Maps the set of extended fingers to a gesture name.
`draw_hands()`	Draws skeleton, dots, label and the hold progress bar.

emotion.py — facial expression

Function	Responsibility
`EmotionEngine.__init__()`	Loads the face model with blendshape output enabled.
`EmotionEngine.detect()`	Returns dominant expression, all scores, and the face box in pixels.
`EmotionEngine._scores()`	Converts 52 coefficients into six normalised scores, with a neutral floor so weak signals do not read as anger.

depth.py — metric depth

Function	Responsibility
`DepthEngine.__init__()`	Loads the metric model onto the GPU.
`DepthEngine.estimate()`	Returns a float map in metres at frame resolution.
`DepthEngine.distance_at()`	Median of a small window, so a click does not jitter on one noisy pixel.
`DepthEngine.colourise()`	Turbo heatmap clipped to 0.5–8 m.
`DepthEngine.pointcloud()`	Back-projects the depth grid with 60-degree intrinsics into x,y,z,r,g,b.

4. HTTP interface

Endpoint	Returns
`GET /`	Dashboard page.
`GET /stream`	MJPEG stream of the annotated frame.
`GET /snapshot`	One annotated JPEG, or the stopped placeholder.
`GET /events`	Stats, both feeds, gesture state, expression, objects, feature flags.
`GET /measure?x=&y=`	Distance in metres at normalised coordinates.
`GET /depthmap`	Depth heatmap JPEG.
`GET /pointcloud`	Binary float32 cloud, six values per point.
`POST /api/toggle`	Enable or disable depth, cloud, emotion, objects.
`POST /api/run`	Start or stop capture. Stop releases the device.

5. Measured performance

Stage	Cost on the RTX 4060 laptop
Person detection and tracking	5–6 ms per frame
Hand landmarks, 0.6 scale, every other frame	10–15 ms
Expression scoring	~10 ms, CPU only
Metric depth	~300 ms per pass
Scene narration	1.6–1.9 s warm
End to end, hands enabled	~40 fps
End to end, tracking only	~60 fps

Memory: about 1.4 GB of VRAM for the depth model, shared with a 2.9 GB local vision model that the system loads only when narration is used.

6. Known limits

- **Glass.** Monocular depth has no cue for a transparent pane. It reports the surface behind it, or invents one. Wall and furniture distances are usable; window distances are not.
- **Expression is not mood.** The classifier reads the face. Glasses, beards, side lighting and laptop cameras all reduce accuracy.
- **3D view is a sketch.** There is no loop closure and no drift correction. Geometry wobbles as depth is re-estimated. Accurate mapping needs a depth sensor and a SLAM stack.
- **Class, not instance.** The detector reports categories from the 80-class COCO set. Recognising your own objects requires training a model on your data.

- **One camera at a time.** The device is exclusive by design.

7. Running it

```
bash setup.sh          # venv, torch, ultralytics, opencv (once)
./svc.sh start        # or: python webui.py --port 8099
./svc.sh status
./svc.sh stop
```

Open `http://127.0.0.1:8099`. Start and Stop in the header control capture; Stop releases the camera so other software can use it. Toggles add distance, a 3D point cloud, expression and full object detection.

> Training your own detector needs 20–50 labelled frames per class and runs on the same GPU: `yolo train model=yolo11n.pt data=data.yaml epochs=100 imgsz=640 batch=8`.